

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ "ЛЬВІВСЬКА ПОЛІТЕХНІКА"



**АВТОМАТИЗОВАНЕ ПРОЕКТУВАННЯ КОМП'ЮТЕРНИХ
СИСТЕМ**

Лабораторна робота №4

Створення Doxygen документації.

Виконав: ст. гр. КІ-404
Стефанів В.В.
Прийняв:
Федак П.Р.

Львів 2024

Мета роботи: Написати коментарі документації Doxygen для всіх публічних сутностей та згенерувати документацію.

ТЕОРЕТИЧНІ ВІДОМОСТІ

Doxygen і коментарі документації є важливими інструментами для створення зрозумілої, підтримуваної, та детальної документації коду в мовах програмування, таких як TypeScript та C++. У звіті можна включити наступні аспекти:

Doxygen

Doxygen — це популярний інструмент для автоматичної генерації документації з коментарів, вставлених у вихідний код. Він підтримує C++, C, Java, Python та інші мови, а також може бути налаштований для TypeScript. Doxygen дозволяє генерувати документацію у вигляді HTML, PDF та інших форматів, що значно спрощує пошук необхідної інформації. Основними перевагами Doxygen є:

1. **Автоматизація документації:** Замість створення окремих файлів для документації, розробники можуть вставляти коментарі у код, а Doxygen автоматично згенерує їх у зручному форматі.
2. **Підтримка різних форматів документації:** HTML, LaTeX, Man pages, RTF тощо.
3. **Зручний навігаційний інтерфейс:** Doxygen створює індексацію класів, методів та змінних, що дозволяє легко знаходити інформацію про конкретні елементи коду.
4. **Гнучкість:** Інструмент підтримує широкий набір тегів та маркувань для різних мов програмування.

Основні теги Doxygen для C++:

- `@brief` — короткий опис функції, класу або змінної.
- `@param` — описує параметри функції.
- `@return` — описує, що повертає функція.
- `@see` — посилання на інші пов'язані класи або функції.
- `@code ... @endcode` — блок коду, який додається до документації.

Коментарі документації у TypeScript

TypeScript також підтримує коментарі документації, що полегшує роботу з IDE, такими як Visual Studio Code, де документація підсвічується безпосередньо при наведенні курсору на метод чи змінну. Основні теги аналогічні до тих, що використовуються в Doxygen, і включають:

- `@param` — описує параметри функції.
- `@returns` — описує, що повертає функція.

- `@deprecated` — вказує, що метод застарілий.
- `@example` — наводить приклад використання функції або методу.

ЗАВДАННЯ

1. Додати коментарі Doxygen коментарі для всіх публічних функцій, класів, полів...
2. Згенерувати документацію на основі коментарів Doxygen.
3. Обов'язкові кроки до кожної лабораторної.

Task 4. Create doxygen documentation:

1. Add doxygen comments for all public functions, classes, properties, fields...
2. Generate documentation based on doxygen comments
3. Required steps

Виконання роботи

1. Написав коментарі документації для серверної частини (arduino). Код для arduino написаний мовою C++ та міститься тільки в одному файлі.

```
/**
 * @class GameState
 * @brief Manages the game state, including scores, game mode, and recent moves.
 */
class GameState {
public:
    int player1Score = 0;
    int player2Score = 0;
    String gameMode = "hvh"; // hvh, hvc, cvc, cvcs
    String lastMove1 = "";
    String lastMove2 = "";
    String lastComputerMove = "";
};

/**
 * @class GameStrategy
 * @brief Defines a strategy interface for generating moves in different game modes.
 */
class GameStrategy {
public:
    /**
     * Generates a move for the game.
     * @return A string representing the move ("rock", "paper", or "scissors").
     */
    virtual String makeMove() = 0;
};
```

Рис.1. Приклад документування коду серверної частини.

2. Створив файл Doxyfile з конфігурацією Doxygen для максимальної підтримки

C++ та відповідно до наших потреб.

```
# Project details
PROJECT_NAME           = "Rock Paper Scissors Server"
OUTPUT_DIRECTORY       = ./docs
INPUT                  = server.ino
FILE_PATTERNS          = *.ino
EXTENSION_MAPPING      = ino=C++
OPTIMIZE_OUTPUT_FOR_C  = YES

# Extract everything
EXTRACT_ALL            = YES
EXTRACT_PRIVATE        = YES
EXTRACT_STATIC         = YES

# Graphics settings
HAVE_DOT               = YES
CLASS_DIAGRAMS         = YES
CLASS_GRAPH            = YES

# Output settings
GENERATE_HTML          = YES
GENERATE_LATEX         = NO

# Source code settings
SOURCE_BROWSER         = YES
INLINE_SOURCES         = YES

# Arduino specific settings
BUILTIN_STL_SUPPORT    = YES
TYPEDEF_HIDES_STRUCT  = YES

# Documentation settings
JAVADOC_AUTOBRIEF     = YES
QT_AUTOBRIEF           = YES
MULTILINE_CPP_IS_BRIEF = YES
INHERIT_DOCS           = YES
DISTRIBUTE_GROUP_DOC   = YES
```

Рис.2. Doxyfile серверної частини

3. Згенерував документацію серверної частини використовуючи команду doxygen Doxyfile.



Рис.3. Приклад згенерованої документації серверної частини

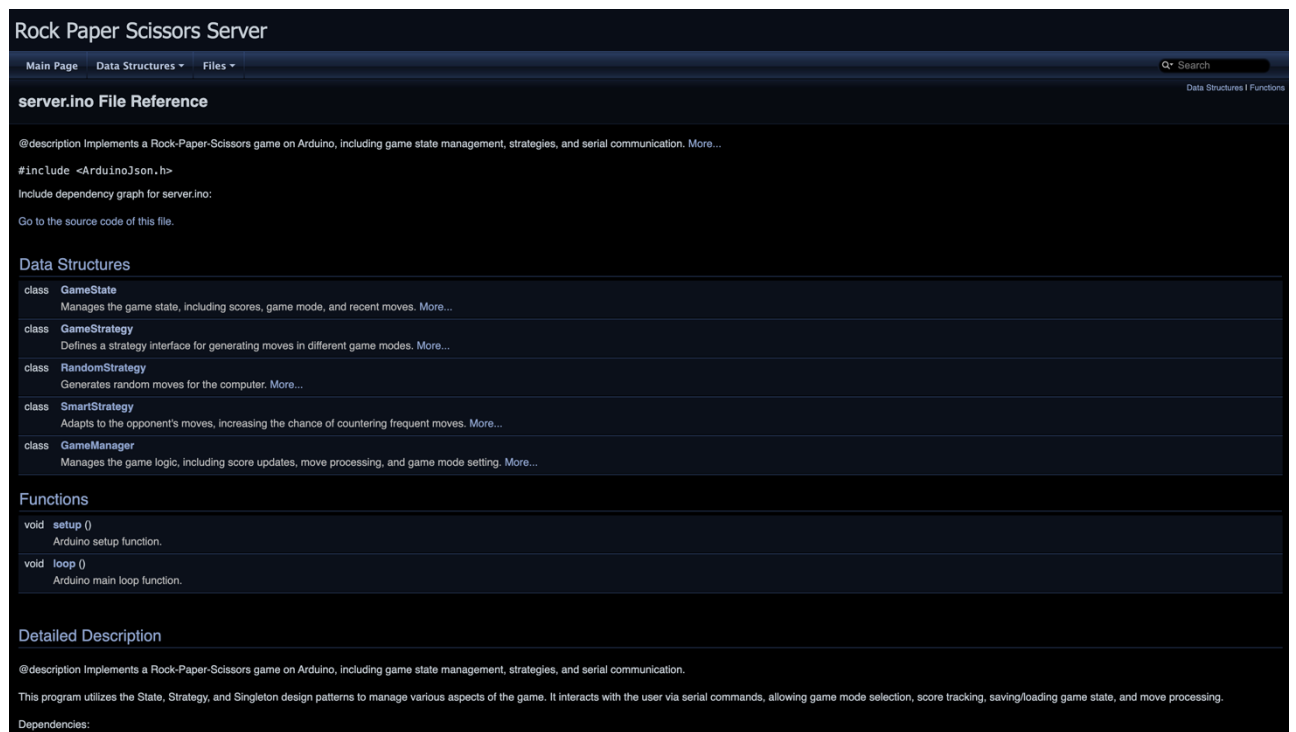


Рис.4. Приклад згенерованої документації серверної частини

4. Написав коментарі документації для клієнтської частини. Код для цієї частини додатку написаний мовою TypeScript, яку нативно Doxygen не

підтримує. Для того щоб обійти це обмеження, використовуємо коментарі JsDoc та налаштовуємо Doxyfile під максимальну підтримку мови програмування Java, яка досить схожа на TypeScript.

```
/**
 * @interface ButtonProps
 * @brief Button properties that extend the default HTML button attributes.
 *
 * @property variant Specifies the style variant of the button.
 *                It can be "primary", "secondary", or "danger".
 */
interface ButtonProps extends React.ButtonHTMLAttributes<HTMLButtonElement> {
  variant?: "primary" | "secondary" | "danger";
}

/**
 * @class Button
 * @brief A factory pattern implementation to create a customizable Button component.
 *
 * @param children The content to display within the button.
 * @param variant The button's style variant, defaulting to "primary".
 * @param className Additional custom class names for styling.
 * @param props Additional properties passed to the HTML button element.
 *
 * @return A styled button component based on the specified variant.
 */
export const Button: React.FC<ButtonProps> = ({
  children,
  variant = "primary",
  className = "",
  ...props
}) => {
  /**
   * @brief Determines the CSS class for the button variant.
   *
   * @return A string representing the CSS class for the chosen variant.
   */
  const getVariantClass = () => {
    switch (variant) {
      case "secondary":
        return styles.secondary;
      case "danger":

```

Рис.5. Приклад документування коду серверної частини.

5. Створив файл Doxyfile з конфігурацією Doxygen для клієнтської частини з максимальною підтримкою Java, TypeScript та відповідно до наших потреб.

```

# Project settings
PROJECT_NAME      = "Rock Paper Scissors Client"
OUTPUT_DIRECTORY  = docs

# Input settings
INPUT             = ./src
FILE_PATTERNS     = *.ts *.tsx
RECURSIVE         = YES

# TypeScript specific settings
EXTENSION_MAPPING = ts=JavaScript tsx=JavaScript
OPTIMIZE_OUTPUT_JAVA = YES
JAVADOC_AUTOBRIEF  = YES

# Enable all supported cross-reference formats
REFERENCED_BY_RELATION = YES
REFERENCES_RELATION     = YES

# Enable preprocessing
ENABLE_PREPROCESSING = YES
MACRO_EXPANSION      = YES
EXPAND_ONLY_PREDEF   = NO

# Handle TypeScript/JavaScript constructs
EXTRACT_ALL          = YES
EXTRACT_PRIVATE      = YES
EXTRACT_STATIC       = YES
EXTRACT_LOCAL_CLASSES = YES

# Source browsing
SOURCE_BROWSER        = YES
INLINE_SOURCES        = YES

# Output settings
GENERATE_HTML         = YES
GENERATE_LATEX        = NO

# Include all documentation
FULL_PATH_NAMES       = YES
STRIP_FROM_PATH       = ./src

# Handle modern JavaScript
OPTIMIZE_FOR_TYPESCRIPT = YES
MARKDOWN_SUPPORT       = YES

# Diagrams and graphs
HAVE_DOT              = YES

```

Рис.6. Doxyfile клієнтської частини

6. Згенерував документацію клієнтської частини використовуючи команду doxygen Doxyfile.



Рис.7. Приклад згенерованої документації клієнтської частини

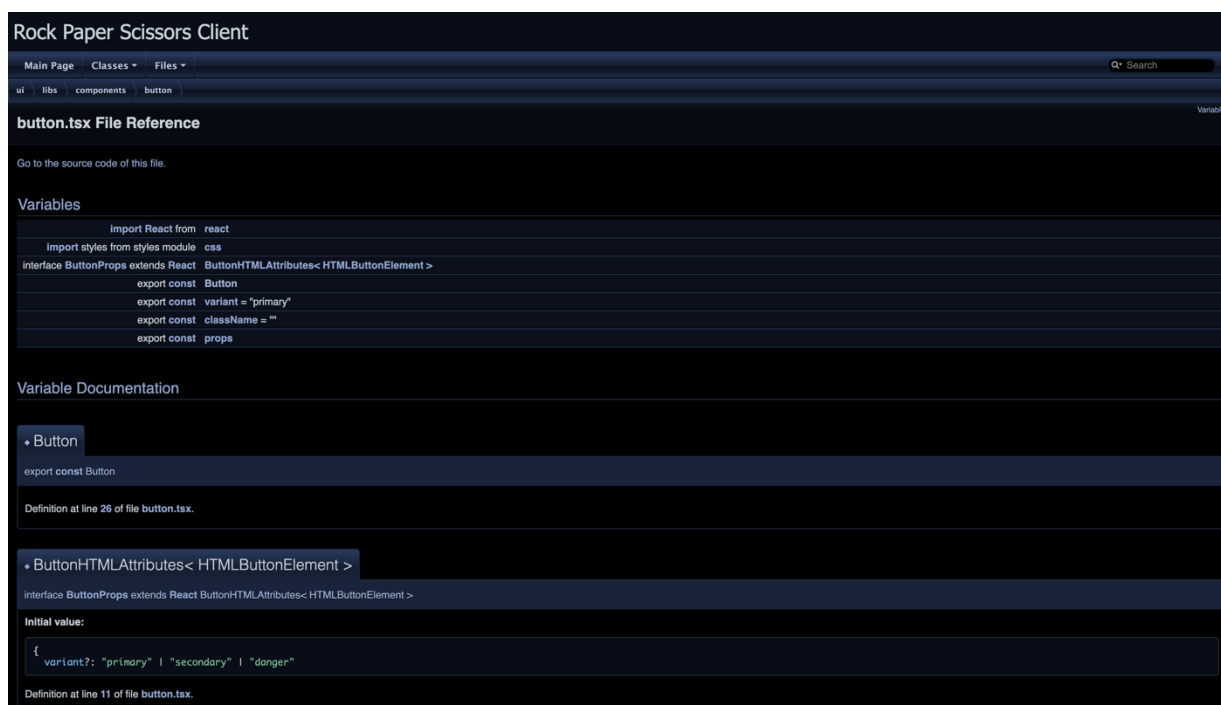


Рис.8. Приклад згенерованої документації клієнтської частини

Висновок: У ході виконання лабораторної роботи я досліджив методи документування коду за допомогою інструменту Doxygen та коментарів документації в TypeScript і C++. Було розглянуто основні теги та принципи створення структурованих коментарів, що значно підвищують зручність використання і підтримки коду. Засвоїв знання про інструмент Doxygen, який дозволяє автоматизувати генерацію документації та створювати її в різних форматах, що є зручним для командної роботи і спрощує ознайомлення з кодом нових розробників.